

Semana 3
ISIS-1611: Inteligencia Artificial

Carla González

16 de junio de 2026

1. Notas de Clase

1.1. Adversaria y juegos

En cuanto a la agenda es:

1. Teoría de juegos: como juegos de cero-suma
2. Decisiones optimas en juegos: algoritmo minimax, decisiones optimas en juegos multijugador, podada alfa-beta
3. Búsqueda de árbol alpha-beta heurístico: funciones de eval, cortando la búsqueda, podada forward

Las **Relajación de restricciones** es una técnica utilizada para simplificar problemas complejos de búsqueda y optimización. Consiste en *eliminar o debilitar* algunas de las reglas de un problema para encontrar rápidamente una **solución aproximada**. Los pasos son los siguientes:

- saber definir las variables y los dominios de valores
- saber definir las restricciones entre variables
- conocer la forma de resolución de este tipo de problemas y posibles heurísticas

Un ejemplo claro es el del sudoku, en este caso hay 8 reinas(8 variables) cada una tiene su posición (x, y) cuyo dominio entero representa su posición en la fila/columna 1..8. Cada variable puede tener un **dominio distinto**. Las **restricciones** las reinas no pueden compartir fila ni columna ni diagonal. En el sudoku los números no pueden repetirse en filas ni columnas ni en bloques de 3x3. Pueden involucrar a tantas variables como queramos (binarias, ternarias, etc)

1.2. Como aborda un CSP?

Generamos un espacio de búsqueda, y analizamos cada variable/mejorar la complejidad: Hay un conjunto de **heurísticas** con distintas aproximaciones (*tecnicas de inferencia*) y durante la búsqueda *propagación y consistencia* y *criterios de ordenación y selección de variables y valores a instanciar*

Util cuando tenemos un problema que se puede modelar como combinación de variables y restricciones. $x_1, x_2 \in [1..10]$ donde $x_1 + x_2 > 10$ y $x_1 - x_2 < 5$. Nos interesa encontrar cualquier solución que satisfaga todas las restricciones - satisfactibilidad. Proceso de resolución muy caro. Elevada explosión combinatoria de heurísticas.

Los tipos de problemas de estados, se presentan mediante **estado inicial**, un conjunto de acciones o transiciones y un estado objetivo. Se clasifican principalmente en 4 tipos según el entorno, la información disponible y el número de estados simultáneos que el agente maneja.

1. **Problemas de un solo estado:** el agente asume que el entorno es totalmente observable y determinista. Conoce exactamente el estado actual y el resultado de cada acción.
2. **Problemas de estados múltiples:** el entorno no es totalmente observable. El agente no sabe en qué estado se encuentra exactamente. Debe mantener un conjunto de *estados posibles* y actualizarlo paso a paso hasta confirmar su posición.
3. **Problemas de contingencia:** el entorno es dinámico, hostil o incluye a otros agentes. El plan inicial puede fallar porque ocurren eventos impredecibles. El agente debe calcular planes flexibles y modificarlos en tiempo real según la info que percibe del entorno.
4. **Problemas de exploración:** el agente no tiene conocimiento previo de los estados ni de las reglas del entorno. Debe interactuar directamente con el para descubrir los estados, acciones y los costos.

Para **Problemas discretos**: Hill-climbing search, Simulated annealing, local beam search, algoritmos genéticos. **Problemas continuos**: Estrategias. **Problemas no determinísticos**: AND-OR Search trees, Try, Try again. **Problemas no observables**: agentes en ambientes no observables, agentes en ambientes parcialmente observables. Y por ultimo los ambientes desconocidos es busqueda online vvs offline.

1.2.1. Optimización de estrategias

Está la **maximización de beneficios** donde los modelos ayudan a los agentes de IA a identificar estrategias que maximicen sus beneficios en diversos escenarios, garantizando que tomen las decisiones mas ventajosas. Y el **equilibrio entre ventajas y desventajas** donde en situaciones que implican multiples objetivos, la teoria de juegos ayuda a equilibrar las ventajas y desventajas para lograr el mejor resultado general.

2. Busqueda Adversaria

2.1. Juegos de Cero Suma

La ganancia del jugador es la pérdida del otro.

2.2. Minimax en juegos de suma cero

Minimax es una regla de decision para minimizar la perdida posible en el peor de los casos. En los juegos de suma cero, la ganancia de un jugador supone la pérdida de otro. Enfoque donde se empieza desde los sucesores y si se escoge el valor maximo se toma el mayor valor.

```

1  function minimax(position, depth, maximizingplayer):
2      if depth == 0 or game over in position
3          return static evaluation of position
4      if maximizingplayer:
5          maxeval = -infinity
6          for each child of position:
7              eval = minimax(child, depth - 1, false)
8              maxeval = max(maxeval, eval)
9          return maxeval
10     else:
11         mineval = +infinity
12         for each child of position:
13             eval = minimax(child, depth - 1, true)
14             mineval = min(mineval, eval)
15     return mineval

```

2.3. Poda alfa-beta

Minimizar la carga computacional necesaria para explorar un arbol de juego, excluyendo las ramas que con certeza no serán optimas.

α representa la puntuación maxima encontrada a lo largo del camino para el agente de IA.
 β representa la puntuación minima para el oponente

A medida que avanza la busqueda, cuando el algoritmo identifica un movimiento que conduce a una situacion peor que la conocida, elimina esa rama, ya que es irrelevante.

```

1  function minimax(position, depth, alpha, beta, maximizingplayer):
2      if depth == 0 or game over in position
3          return static evaluation of position
4      if maximizingplayer:

```

```

5         maxeval = -infinity
6         for each child of position:
7             eval = minimax(child, depth - 1, alpha, beta, false)
8             maxeval = max(maxeval, eval)
9             alpha = max(alpha, eval)
10            if beta <= alpha
11                break
12        return maxeval
13    else:
14        mineval = +infinity
15        for each child of position:
16            eval = minimax(child, depth - 1, alpha, beta, true)
17            mineval = min(mineval, eval)
18            beta = min(beta, eval)
19            if beta <= alpha
20                break
21        return mineval

```

Es una extensión y hay relajación de restricciones. Juegos estocásticos Los juegos clásicos nosotros estamos viendo es si las maquinas pueden jugar? Puedan tomar mejores decisiones que nosotros? Los juegos son simulaciones, ambientes parcialmente observables, tiene un numero de estados bastante grandes.

2.4. Procesos de decisión markovianas

2.5. Tipos de juegos

Tipo de objetivos (First person S) Hay distintos tipos de ejes:

- Determinístico o estocásticos
- uno, dos o mas jugadores

Definición 2.1. Tenemos una utilidad que $U=A,B$, donde A maximiza y B minimiza

Cooperación implícita En los juegos generales los agentes tienen utilidades independientes, cooperación indiferencia competencia ->posibles

Agente toma decisiones según el punto de decisión, la utilidad es como la recompensa

2.6. Algoritmo MiniMax

Se aplica para juegos determinísticos y de cero suma. Hay un árbol de espacio de estados. Jugador alterna turnos. Computa el valor minimax de cada nodo (Mejor Utilidad alcanzable contra un jugador racional)

Recompensas $\neq$ Búsqueda

$U = f(n)$ $g =$ camino recorrido y $h =$ heurística

2.6.1. Decisiones óptimas en Juegos

Casos de cooperación implícita, donde hay múltiples jugadores se toma en cuenta el costo por acción y considerar que hace al agente adversario

2.7. Límites de Recursos

Son juegos realísticos no puede buscar a hojas La solución es Depth-limited search, lo cual busca solucionar para un espacio limitado, reemplazar las utilidades terminales con una función de evaluación para posiciones no terminales Podada Forward, uso de estadísticas. Algoritmo de corte probabilístico y reducción de movimientos tardíos